

Buscador Semántico Simple

Proyecto de Investigación 2 - Álgebra Lineal para la Computación

Denys Rodríguez, Paulo Villalobos, Joaquín Valenzuela, David Villegas

21 de junio de 2026

Índice

1. Representación de palabras y documentos como vectores	3
1.1. El Espacio Vectorial Textual	3
1.1.1. El vocabulario y la dimensión del espacio	3
1.1.2. Representación numérica de palabras y documentos	3
1.1.3. Estructura y restricciones algebraicas	4
1.1.4. Interpretación geométrica	4
1.1.5. Dimensión del espacio y dispersión (Sparsity)	5
1.2. La Similitud Coseno	5
1.2.1. Coseno frente a la distancia euclidiana	6
1.2.2. Interpretación geométrica del ángulo y rango de valores	6
1.3. Ejemplo numérico desarrollado	6
1.3.1. Preprocesamiento y construcción del vocabulario	7
1.3.2. Construcción de los vectores TF	7
1.3.3. Cálculo del producto punto a mano	7
1.3.4. Cálculo de las normas a mano	8
1.3.5. Resultado del coseno y su interpretación	8
2. La Matriz Documento-Término	9
2.1. Construcción de la matriz y dispersión	9
2.2. Análisis del corpus del equipo y orquestación	9

1. Representación de palabras y documentos como vectores

1.1. El Espacio Vectorial Textual

Si queremos que una computadora compare diferentes textos y decida si guardan alguna relación temática, nos enfrentamos al problema de que no es posible procesar las palabras directamente de forma cualitativa. De esta forma, el álgebra lineal nos ofrece una solución al permitirnos transformar cada palabra y cada documento en un vector. Como resultado, los textos pasan a ser puntos dentro de un espacio geométrico de muchas dimensiones, lo que permite obtener una representación matemática útil para medir su similitud.

1.1.1. El vocabulario y la dimensión del espacio

El proceso inicia tomando todos los textos de nuestro corpus (el conjunto de documentos) para someterlos a una limpieza o preprocesamiento que elimina puntuación, pasa todo a minúsculas, remueve acentos y quita palabras vacías como “el”, “un” o “de”. La lista que reúne todas las palabras únicas que sobreviven a esta limpieza termina representando nuestro **vocabulario**, al cual denotamos matemáticamente como:

$$V = \{w_1, w_2, \dots, w_n\}$$

donde n terminaría siendo la cantidad total de palabras distintas que pudimos rescatar.

La dimensión del espacio vectorial en el que vamos a trabajar se define a partir del tamaño de este vocabulario, por lo que diríamos que $\dim(\mathcal{V}) = n$. Detrás de esto se encuentra la idea de que cada palabra única del vocabulario puede interpretarse como una dimensión o eje coordenado independiente. Como resultado, si nuestro vocabulario final contara con 5000 términos únicos, los documentos pasarían a representarse como vectores en el espacio real de 5000 dimensiones, $\mathbb{R}^{5000}$.

1.1.2. Representación numérica de palabras y documentos

Para construir los vectores de cada documento, conviene entender primero cómo podemos estructurar algebraicamente las palabras individuales:

1. **One-Hot Encoding (Codificación con un único uno)**: Es la forma más simple para vectorizar palabras individuales. A cada palabra $w_i \in V$ le asignamos un vector que tiene un valor de 1 en la posición i y ceros en todas las demás posiciones. De esta forma, cada palabra pasa a estar representada por un vector de la base canónica $\mathbf{e}_i$:

$$\mathbf{e}_i = [0, \dots, 0, \underbrace{1}_{\text{pos. } i}, 0, \dots, 0]^T \in \mathbb{R}^n$$

Al usar la base canónica, estamos asumiendo algebraicamente que todas las palabras son ortogonales entre sí. Esto implica que, en este punto, el modelo considera que las palabras son independientes entre sí, por lo que terminaría interpretando que “fútbol” y “balompié” son tan lejanas entre sí como lo serían “fútbol” y “átomo”.

2. **Bag-of-Words (Bolsa de Palabras)**: Consiste en crear el vector de un documento sumando los vectores de cada palabra que contiene. Aunque al hacer esto perdemos el orden de las palabras y la gramática, logramos mantener la información de qué palabras aparecen y cuántas veces lo hacen.
3. **Frecuencia de Términos (TF - Term Frequency)**: Para esta representación, cada documento d_j pasa a ser un vector $\mathbf{d}_j \in \mathbb{R}^n$ donde cada posición muestra cuántas veces aparece cada palabra del vocabulario en el texto. Esto se escribe matemáticamente como:

$$\mathbf{d}_j = [f_{1j}, f_{2j}, \dots, f_{nj}]^T$$

donde f_{ij} terminaría siendo la cantidad de veces que la palabra w_i aparece en el documento d_j .

Significado práctico y normalización: Si usáramos los conteos directos de las palabras, los documentos más largos tendrían números mucho más altos simplemente por ser más largos. Esto produciría vectores con longitudes (normas) muy grandes, lo que afectaría las comparaciones. Por lo tanto, dividimos cada frecuencia entre el total de palabras que tiene el documento:

$$\text{TF}(w_i, d_j) = \frac{f_{ij}}{\sum_{k=1}^n f_{kj}}$$

De esta forma, cada coordenada pasa a mostrar qué tan importante es esa palabra en el texto en relación con las demás, lo que permite compararlos de manera justa.

1.1.3. Estructura y restricciones algebraicas

Aunque modelamos los vectores de los documentos dentro del espacio vectorial real $\mathbb{R}^n$, en la práctica nos topamos con un límite muy claro: no es posible que una palabra aparezca una cantidad negativa de veces en un texto. Por lo tanto, los números de nuestros vectores siempre terminan siendo mayores o iguales a cero ($\text{TF} \geq 0$).

Como resultado, los vectores de documentos no ocupan todo el espacio $\mathbb{R}^n$, sino que se quedan en el **ortante no negativo** (que es el equivalente de más dimensiones al primer cuadrante que conocemos en el plano cartesiano $\mathbb{R}^2$), denotado como $\mathbb{R}_+^n$. Aunque tengamos este límite, usamos las reglas de $\mathbb{R}^n$ porque nos permiten realizar operaciones matemáticas muy útiles:

- **Suma vectorial**: Si sumamos dos vectores de documentos, el vector que obtenemos termina representando la combinación de ambos textos, sumando las palabras que aparecen en ellos.
- **Multiplicación por un escalar**: Si multiplicamos el vector de un documento por un número positivo $\alpha > 0$, cambiamos el largo del vector pero mantenemos las proporciones de sus palabras. De esta forma, el vector mantiene la misma dirección temática, cambiando solamente su tamaño.

1.1.4. Interpretación geométrica

En este modelo (conocido como Modelo de Espacio Vectorial o VSM), cada documento puede interpretarse como una flecha (un vector de posición) que sale del origen $(0, 0, \dots, 0)$ y apunta hacia un punto en este espacio de n dimensiones.

De esta forma, la dirección de la flecha pasa a mostrar de qué habla el documento. Si dos textos usan palabras parecidas en proporciones similares, sus vectores van a apuntar en direcciones muy cercanas, formando un ángulo pequeño. En cambio, si no comparten ninguna palabra, sus vectores terminarán siendo perpendiculares u **ortogonales**, lo que significa que forman un ángulo de 90° .

Esta mirada geométrica nos ayuda a entender por qué la distancia en línea recta (la distancia euclidiana) no funciona bien para comparar textos, ya que cambia mucho según qué tan largo sea el documento. En su lugar, preferimos medir qué tan parecidos son comparando el ángulo de los vectores, lo que nos lleva a usar la similitud coseno.

1.1.5. Dimensión del espacio y dispersión (Sparsity)

El uso de un vocabulario grande trae consigo dos efectos geométricos e informáticos que debemos tener en cuenta:

1. **Alta dimensionalidad:** A medida que sumamos textos al proyecto, la dimensión n del espacio vectorial crece rápidamente hasta tener miles de dimensiones. En espacios tan grandes, ocurre lo que se conoce como la “maldición de la dimensionalidad”, donde las distancias en línea recta pierden su utilidad porque casi todos los vectores parecen estar a una distancia muy parecida entre sí. Como resultado, la dirección (el ángulo) pasa a ser una medida mucho más útil para comparar los textos.
2. **Dispersión (Sparsity):** Aunque el espacio vectorial tenga 10000 dimensiones por el tamaño del vocabulario, un documento común y corriente solo va a usar unas 50 o 100 palabras únicas. Por lo tanto, casi todas las coordenadas del vector $\mathbf{d}_j$ terminarán siendo iguales a cero. De esta forma, obtenemos vectores altamente **dispersos** (o *sparse*). Para la programación, esto es una ventaja enorme, ya que nos permite guardar y procesar únicamente los números distintos de cero, lo que ahorra mucha memoria y tiempo de cálculo.

1.2. La Similitud Coseno

Si ya tenemos los documentos transformados en vectores dentro de nuestro espacio de vocabulario, el siguiente paso es encontrar una forma matemática para medir qué tan parecidos son entre sí. Para esto, la herramienta que nos proporciona la geometría es la similitud coseno. En lugar de comparar los textos por la cantidad total de palabras, esta medida se enfoca en evaluar la dirección de los vectores. La similitud coseno del ángulo θ formado entre dos vectores $\mathbf{u}$ y $\mathbf{v}$ se calcula usando la siguiente relación:

$$\cos(\theta) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$$

donde el numerador $\mathbf{u} \cdot \mathbf{v}$ terminaría representando el producto punto (la suma de los productos de cada coordenada), lo que nos indica las palabras que tienen en común. Por otro lado, el denominador $\|\mathbf{u}\| \|\mathbf{v}\|$ muestra la multiplicación del largo o magnitud de ambos vectores.

1.2.1. Coseno frente a la distancia euclidiana

Una pregunta natural que surge en este punto es por qué preferimos calcular el coseno del ángulo en lugar de usar la distancia en línea recta (distancia euclidiana) para comparar los textos. La razón de esta elección tiene que ver con la longitud de los documentos. Si tuviéramos un documento corto y un documento largo que hablan exactamente del mismo tema (por ejemplo, fútbol), el documento largo usaría las palabras con frecuencias mucho más altas. Como resultado, su vector terminaría siendo mucho más largo, situando su punto muy lejos en el espacio en comparación con el vector del documento corto.

Si usáramos la distancia euclidiana, esta diferencia de largo produciría una distancia muy grande entre ambos puntos, lo que nos llevaría a pensar que son textos muy diferentes, lo que afectaría la precisión de la búsqueda. Al usar la similitud coseno, el cálculo ignora por completo el largo de los vectores y evalúa únicamente la dirección de las flechas. De esta forma, logramos comparar los textos por su temática sin importar si uno es mucho más largo que el otro.

Nota sobre la normalización: Dado que la fórmula de la similitud coseno ya divide el producto punto por el largo de los vectores, realiza su propia normalización geométrica. Esto significa que el resultado final del coseno será exactamente idéntico ya sea que calculemos la fórmula usando conteos crudos de palabras o usando las frecuencias relativas (TF) previamente normalizadas. Sin embargo, decidimos aplicar la normalización TF en un paso previo porque nos permite que los vectores individuales tengan un significado por sí mismos (mostrando la proporción de cada palabra en el texto) y para mantener la coherencia con el procesamiento interno del código del buscador.

1.2.2. Interpretación geométrica del ángulo y rango de valores

El ángulo θ entre los vectores nos da una interpretación geométrica muy directa de la relación entre los documentos, lo que se traduce en un rango de valores específico:

- **Coseno igual a 1:** Ocurre cuando el ángulo es de 0° , lo que implica que ambos vectores apuntan exactamente en la misma dirección. Este valor máximo indica que los documentos comparten las mismas palabras en exactamente las mismas proporciones.
- **Coseno igual a 0:** Ocurre cuando el ángulo es de 90° . En este caso, los vectores terminan siendo perpendiculares u **ortogonales**, lo que significa que los documentos no comparten ni una sola palabra de nuestro vocabulario y, por lo tanto, no tienen ninguna relación temática.
- **Coseno igual a -1:** Ocurre cuando el ángulo es de 180° , lo que indicaría que los vectores apuntan en direcciones opuestas. Sin embargo, dado que en el análisis de textos trabajamos en el ortante no negativo $\mathbb{R}_+^n$, es imposible que una palabra tenga una frecuencia negativa. Como resultado, los vectores nunca pueden apuntar en direcciones opuestas, lo que significa que la similitud coseno en nuestro buscador semántico queda limitada estrictamente al rango entre 0 y 1.

1.3. Ejemplo numérico desarrollado

Para ilustrar de forma práctica cómo funcionan estos conceptos de álgebra lineal en la comparación de textos, vamos a desarrollar un ejemplo completo paso a paso usando dos documentos muy sencillos creados para este fin:

- Documento 1 (d_1): “el gato come pescado”
- Documento 2 (d_2): “el perro come carne”

1.3.1. Preprocesamiento y construcción del vocabulario

Antes de construir los vectores, debemos limpiar los documentos. En este caso, eliminamos la palabra “el” por ser una palabra vacía (stopword) que no aporta significado temático. Como resultado de esta limpieza, los textos quedan reducidos a los siguientes términos:

- Documento 1 (d_1): {gato, come, pescado}
- Documento 2 (d_2): {perro, come, carne}

Al reunir todas las palabras únicas de ambos textos y ordenarlas alfabéticamente, obtenemos el vocabulario V :

$$V = \{\text{carne, come, gato, perro, pescado}\}$$

donde el tamaño de nuestro vocabulario terminaría siendo $n = 5$. Esto implica que trabajaremos en un espacio vectorial de cinco dimensiones, $\mathbb{R}^5$.

1.3.2. Construcción de los vectores TF

Con el vocabulario ordenado, pasamos a representar cada documento como un vector, contando la presencia de cada palabra y normalizando el resultado según el total de términos de cada documento (que en ambos casos es igual a 3):

Para el Documento 1 (d_1), las palabras “come”, “gato” y “pescado” aparecen una vez, mientras que “carne” y “perro” no aparecen. De esta forma, el vector normalizado $\mathbf{d}_1$ pasa a ser:

$$\mathbf{d}_1 = \left[0, \frac{1}{3}, \frac{1}{3}, 0, \frac{1}{3}\right]^T$$

Para el Documento 2 (d_2), las palabras “carne”, “come” y “perro” aparecen una vez, mientras que “gato” y “pescado” no aparecen. Esto nos permite generar el vector normalizado $\mathbf{d}_2$:

$$\mathbf{d}_2 = \left[\frac{1}{3}, \frac{1}{3}, 0, \frac{1}{3}, 0\right]^T$$

1.3.3. Cálculo del producto punto a mano

Para calcular el producto punto entre ambos vectores, multiplicamos sus componentes correspondientes y sumamos los resultados:

$$\begin{aligned}\mathbf{d}_1 \cdot \mathbf{d}_2 &= \left(0 \times \frac{1}{3}\right) + \left(\frac{1}{3} \times \frac{1}{3}\right) + \left(\frac{1}{3} \times 0\right) + \left(0 \times \frac{1}{3}\right) + \left(\frac{1}{3} \times 0\right) \\ \mathbf{d}_1 \cdot \mathbf{d}_2 &= 0 + \frac{1}{9} + 0 + 0 + 0 = \frac{1}{9}\end{aligned}$$

El producto punto da como resultado $\frac{1}{9}$, lo que muestra la coincidencia de la palabra “come” (la única coordenada donde ambos vectores tienen valores distintos de cero).

1.3.4. Cálculo de las normas a mano

Para calcular la norma (o longitud) de cada vector, elevamos al cuadrado cada componente, sumamos estos valores y obtenemos la raíz cuadrada del resultado:

Para el vector $\mathbf{d}_1$:

$$\|\mathbf{d}_1\| = \sqrt{0^2 + \left(\frac{1}{3}\right)^2 + \left(\frac{1}{3}\right)^2 + 0^2 + \left(\frac{1}{3}\right)^2} = \sqrt{\frac{1}{9} + \frac{1}{9} + \frac{1}{9}} = \sqrt{\frac{3}{9}} = \sqrt{\frac{1}{3}} = \frac{1}{\sqrt{3}} \approx 0,577$$

Para el vector $\mathbf{d}_2$, el procedimiento es el mismo al tener la misma cantidad de palabras:

$$\|\mathbf{d}_2\| = \sqrt{\left(\frac{1}{3}\right)^2 + \left(\frac{1}{3}\right)^2 + 0^2 + \left(\frac{1}{3}\right)^2 + 0^2} = \sqrt{\frac{1}{9} + \frac{1}{9} + \frac{1}{9}} = \sqrt{\frac{3}{9}} = \sqrt{\frac{1}{3}} = \frac{1}{\sqrt{3}} \approx 0,577$$

1.3.5. Resultado del coseno y su interpretación

Una vez calculados el producto punto y las normas, usamos la fórmula de la similitud coseno para obtener el valor final:

$$\cos(\theta) = \frac{\mathbf{d}_1 \cdot \mathbf{d}_2}{\|\mathbf{d}_1\| \|\mathbf{d}_2\|} = \frac{\frac{1}{9}}{\left(\frac{1}{\sqrt{3}}\right) \times \left(\frac{1}{\sqrt{3}}\right)} = \frac{\frac{1}{9}}{\frac{1}{3}} = \frac{3}{9} = \frac{1}{3} \approx 0,333$$

El valor obtenido para la similitud coseno termina siendo exactamente $\frac{1}{3} \approx 0,333$. Dado que el rango de la similitud coseno va desde 0 hasta 1 en nuestro espacio de documentos, este resultado muestra una coincidencia baja o moderada entre los textos.

Geométricamente, esto significa que los vectores están separados por un ángulo de aproximadamente $70,5^\circ$ ($\arccos(0,333) \approx 70,5^\circ$). Desde el punto de vista del significado, el resultado es coherente: ambos documentos comparten únicamente la acción (la palabra “come”), pero difieren en el sujeto (“gato” frente a “perro”) y en el objeto (“pescado” frente a “carne”). De esta forma, el modelo matemático es capaz de identificar que los textos guardan cierta relación, pero que no son idénticos.

2. La Matriz Documento-Término

2.1. Construcción de la matriz y dispersión

Una vez que hemos representado cada documento individual como un vector matemático, podemos agrupar todos los textos de nuestro corpus en una sola gran estructura algebraica conocida como la matriz documento-término. Esta matriz se construye alineando los vectores de los documentos uno debajo del otro. Como resultado, obtenemos una matriz bidimensional donde cada fila representa un documento completo y cada columna representa una palabra única del vocabulario.

Si llamamos m a la cantidad de documentos y n a la cantidad de palabras del vocabulario, la matriz resultante tendrá un tamaño de $m \times n$.

Un fenómeno matemático muy particular que ocurre con esta matriz es su alta **dispersión** (o *sparsidad*). Dado que cualquier documento típico utiliza apenas una pequeña fracción de todas las palabras disponibles en el vocabulario general (la mayoría de las palabras de un diccionario no aparecen en un texto corto), casi todos los valores de los vectores serán exactamente cero. Mientras más grande sea el corpus de textos, más palabras nuevas se agregarán al vocabulario general, haciendo que la cantidad de ceros en la matriz crezca enormemente.

2.2. Análisis del corpus del equipo y orquestación

Para poner a prueba nuestro buscador semántico, el equipo de investigación reunió un corpus de 16 documentos cortos. Estos textos se dividen en cuatro temáticas principales: historia, matemáticas, naturaleza y tecnología (cuatro textos por cada tema).

Al aplicar el proceso de limpieza y extracción de palabras (preprocesamiento) sobre los 16 archivos de texto, nuestro sistema detectó un vocabulario exacto de 250 palabras únicas. Con estos datos, la matriz documento-término generada por el programa terminó teniendo un tamaño de 16×250 . Al analizar internamente los valores de esta matriz, observamos que el nivel de dispersión alcanzó un 92.80 %; es decir, más de nueve de cada diez celdas en la matriz albergan un cero.

El proceso completo para leer los archivos, limpiarlos y construir la matriz de frecuencias se puede observar en el siguiente fragmento de código de nuestro proyecto:

```
corpus_files = glob.glob('data/corpus/*.txt')
documentos = []

for filepath in corpus_files:
    with open(filepath, 'r', encoding='utf-8') as f:
        texto = f.read()
        tokens = preprocess(texto)
        documentos.append(tokens)

vocabulario = crear_vocabulario(documentos)
matriz_documento_termino = crear_matriz_tf(documentos, vocabulario)
```